Deep Learning

Part A : Theory

1. Deep Learning Intro
2. Why DL is important
3. Perceptron
4. Activation Functions
5. Layers in Perceptron
6. Single Layered Perceptron Model
7. Multi Layered Perceptron Model
8. Back and weight updation formula
9. Chain rule of derivation
10. Vanishing Gradient Problem
11. Activation functions
    1. Sigmoid function
    2. TanH function
    3. Relu function
    4. Relu (leaky) and Parametric relu
    5. Exponential linear units
    6. Softmax function

12. Which activation function to use
13. Lost and cost function

14. Types of loss function
    1. MSE
    2. MAe
    3. Huber loss
    4. RMSE

15. Loss or Cost Functions
    1. Binary Cross Entropy
    2. Categorical Cross Entropy
    3. Sparse Categorical Cross Entropy

16. Activation Functions Combination with Loss Fucntions

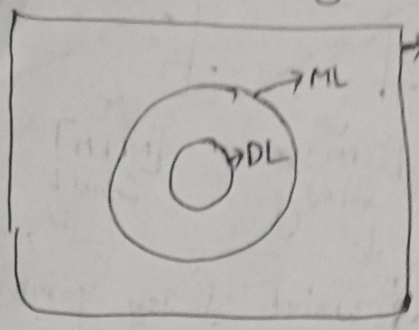
17. Optimizers
    1. Gradient Descent
    2. Stochastic Gradient Descent
    3. Mini Batch SGD
    4. SGD with Momentum
    5. Adagrad Optimizers (Adaptive GD)
    6. Adadelta & RMSPROP
    7. Adam Optimizers

18. Exploding Gradient Descent
19. Weight Initialization Techiniques
    1. Uniform Distribution
    2. Xavier / Glovot Initilization
    3. Kaiming He Initialization

20. DropOut Layer
21. CNN
22. Convolution Operations In CNN
    1. Pooling
        - Max Pooling
        - Mean Pooling
        - Mean Pooling
    2. Fully Connected Layers in CNN
    3. Features Extraction

23. Images

Intro ⟺ Deep Learning



DL
- ANN — classification, Regression
- CNN — (I/p: Image/video)
  - Ex: RCNN, Masked RCNN, Detectron, Yolo, V5, V6, V7
- RNN → (NLP)

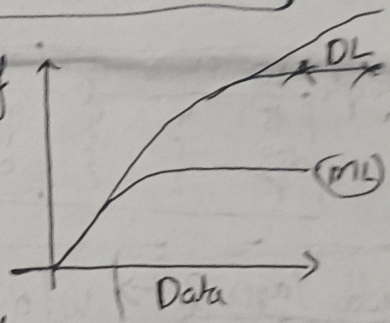
Path ⟩ DL → RNN → [word emb ... | Bidirectional LSTM RNN, Encoder, Decoder] → Transformers

Framework → Tensorflow → End to End project

⟺ Why DL is Important

- History 2005 → Establishment of many Social Media [data was generating exponentially]
  - Bigdata
  - img/vidio → HADOOP → efficient retrieval & Storing
- 2011 | Cloudera, Nostar works
  - GPU cost ↓ by NVidia
- 2012
  - DL gets benefits over backboard
  - DL getting used in many domains
  - Framework Opensource
    - Tensorflow → google
    - Pytorch → facebook

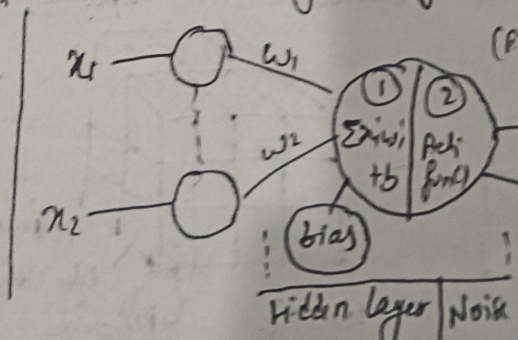
⇒ Community ↑↑ Research ↑↑

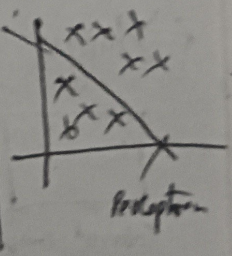


⟺ Perceptron [Artificial Neuron or NN unit]

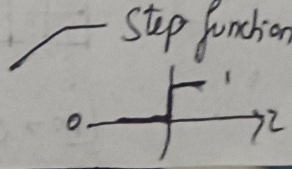
- C_2 classification use | (Input / Hidden) Layer, Weights, Bias, Activation fun()
- Example

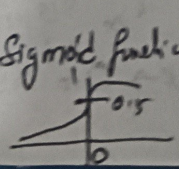
x_1 IQ	x_2 no of Study h	O/P
95	3	F
110	4	P
150	1	P





- Activation fun(z)





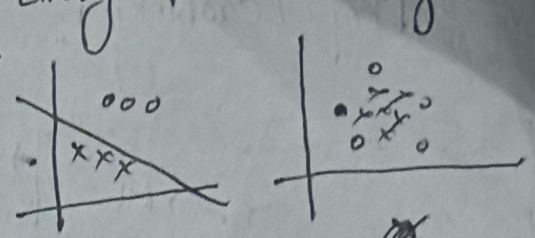
Step fun → $\begin{cases} 0 & z \le 0 \\ 1 & z > 1 \end{cases}$

Sigmoid f ⇒ $\begin{cases} 0 & z < 0.5 \\ 1 & z > 0.5 \end{cases}$

⟺ More in Perceptron

Perceptron
- Single Layer Perceptron Model
- Multi Layered Perceptron Model

Linearly Seperable (2) classification



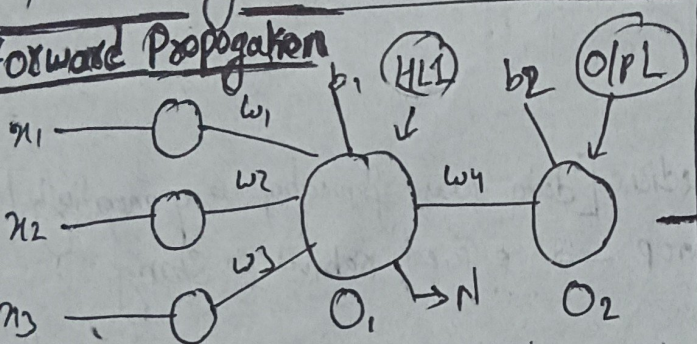
[SLPM] Solved by

[MLPM] Solved by

- Forward Propogation
- Backward Propogation
- Loss function
- Various Activation function
- Ophimizes

⟺ Multi Layered Neuron Network

⇒ Forward Propogation



Example

IQ	Shows	Psych	O/P
95	4	4	1
100	5	2	1
95	2	7	0

$w_i = \begin{bmatrix} 0.01 \\ 0.02 \\ 0.03 \end{bmatrix}$ $b_i = \begin{bmatrix} 0.001 \\ \infty \end{bmatrix}$

Step 1) $Z = \sum_{i=1}^{n} w_i x_i + b$ similar $y = mx + c$

also $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 - \beta_n x_n$

Step 2) Activation fun(z)

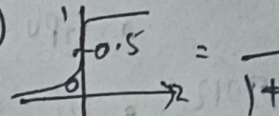
① $Z = x_1 w_1 + x_2 w_2 + x_3 w_3 + b_1$

$Z = \sum_{i=1}^{n} W_i^T x_i + b$

$Z = 95 \times 0.01 + 4 \times 0.02 + 4 \times 0.03 + 1 \times 0.001$

$= 1.151$

② Activation (z) sigmoid



$= \frac{1}{1+e^{-1.151}}$

$O_1 \Rightarrow 0.759$

Hidden Layer → 1

Hidden Layer 2 → $W_2 [0.02]$ $b_2 = [0.03]$

① $Z = 0.04518$

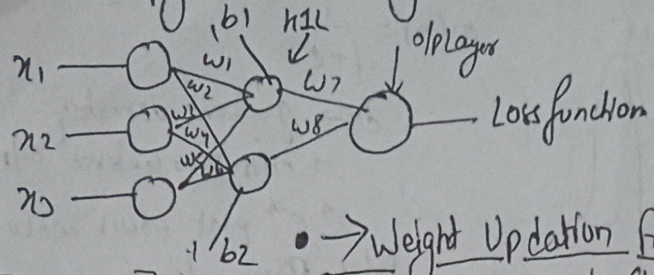
② $act(z) = \boxed{0.51129 = O_2} \Rightarrow \hat{y}$

Loss fun() $= y - \hat{y} = 1 - 0.51129$

$= 0.49$

Backpropogation used backward and updates weights to reduce error

⇔ Back Propogation And Weight Updation formula



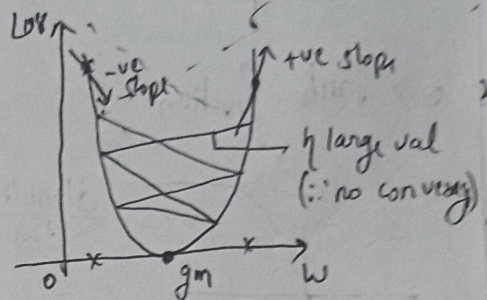
10	Sh	Play h	o/p
95	4	4	1
100	5	2	1
95	2	7	0

$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}_{3\times1} \quad \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \\ w_5 & w_6 \end{bmatrix}_{3\times2} \Rightarrow 3\times2$

- **→ Weight Updation formula**

$$W_{new} = W_{old} - \eta \frac{\delta h}{\partial W_{old}}$$

- **Optimizers** [reduce loss value]

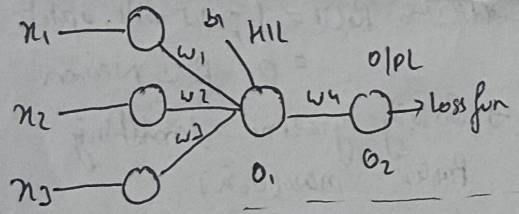


1) $W_{new} = W_{old} - \eta(-ve)$
$= W_{old} + \eta(+ve)$
goal $\boxed{W_{new} \gg W_{old}}$

2) $W_{new} = W_{old} - \eta(+ve)$
$= W_{old} - \eta(+ve)$
goal $\boxed{W_{new} \ll W_{old}}$

Let $\eta = 0.001$ (small val)

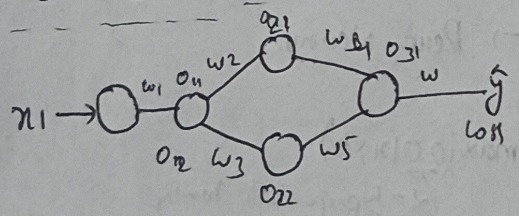
⇔ Chain Rule of Derivation



$$\frac{\partial h}{\partial W_{4old}} = \frac{\partial h}{\partial O_2} * \frac{\partial O_2}{\partial W_{4old}}$$

$$W_{1new} = W_{1old} - \eta \frac{\partial h}{\partial W_{1old}}$$

$$\frac{\partial h}{\partial W_{old}} = \frac{\partial h}{\partial O_2} * \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial W_{1old}}$$

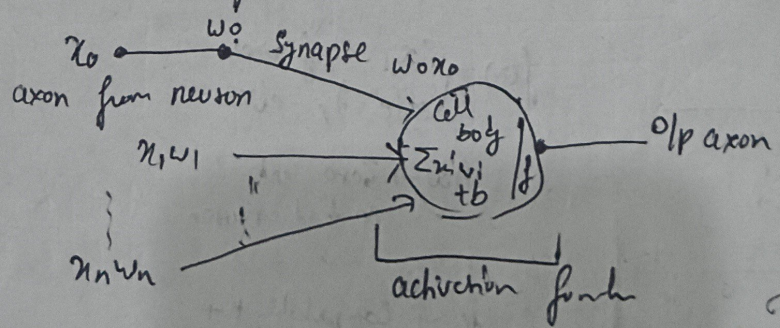


$$W_{1new} = W_{1old} - \eta \frac{\partial h}{\partial W_{1old}}$$

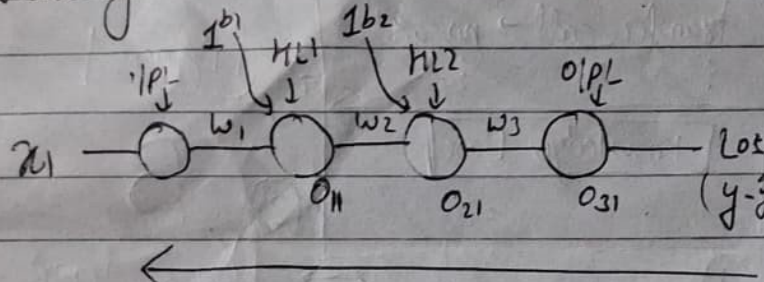
$$\frac{\partial h}{\partial W_{1old}} = \left[\frac{\partial h}{\partial O_{31}} * \frac{\partial O_{31}}{\partial O_{21}} * \frac{\partial O_{21}}{\partial O_{11}} * \frac{\partial O_{11}}{\partial W_{1old}}\right] + \left[\frac{\partial h}{\partial O_{31}} * \frac{\partial O_{31}}{\partial O_{22}} * \frac{\partial O_{22}}{\partial O_{12}} * \frac{\partial O_{12}}{\partial W_{1old}}\right]$$

⇔ Vanishing Gradient Problem

⇔ Activation function



Vanishing Gradient Problem And Activation function

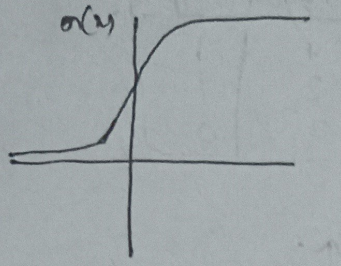


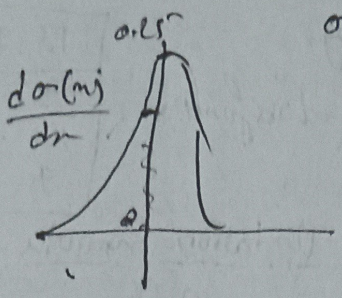
- $w_{1\,new} = w_{1\,old} - \eta \frac{\partial Loss}{\partial w_{1\,old}}$
- $\frac{\partial L}{\partial w_{1\,old}} = \frac{\partial L}{\partial o_{31}} \times \frac{\partial o_{31}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}}$
- $\frac{\partial o_{31}}{\partial o_{21}} = \frac{\partial(\sigma(z))}{\partial(z)} * \frac{\partial z}{\partial o_{21}}$ [chain rule]
- Let $o_{31} = \sigma\left(\frac{w_3 * o_{21} + b_3}{z}\right)$
- Derivative $(\sigma(z)) : 0 \le \sigma(z) \le 0.25$
- $\frac{\partial o_{31}}{\partial o_{21}} = 0 \le \sigma(z) \le 0.25 * w_{3\,old}$

Problem $\Rightarrow$ $w_{1\,new} \approx w_{1\,old}$ (small, small)

Solution $\Rightarrow$ • Tanh • Relu • Prelu • Swish

⇔ Sigmoid function



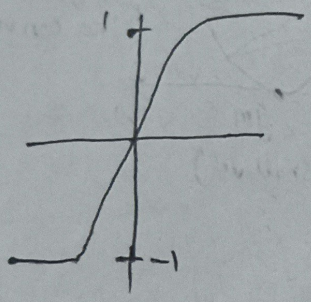


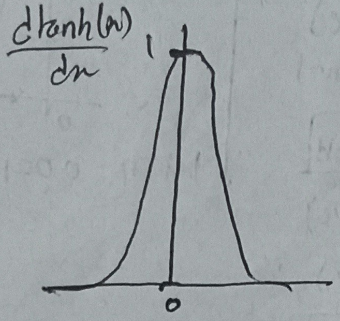
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

dis → gradient vanishing
→ non-zero centered o/p
→ e^{-x} part power consuming

Ad → Binary classification →

⇔ tanh function

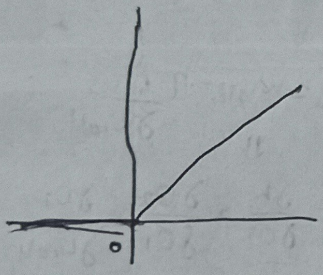


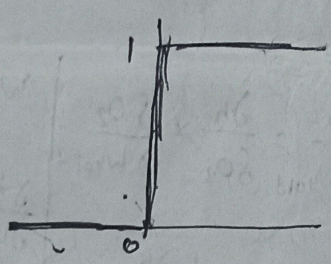


$$\tanh x = \frac{e^{z} - e^{-z}}{e^{z}+e^{-z}}$$

dis → Vanishing gradient E for Deep NN
Computation time consuming

⇔ Relu function





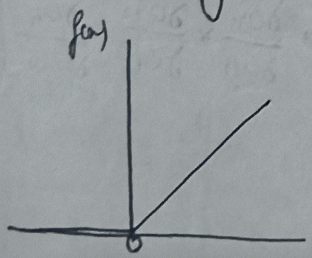
ReLU = max(0, z)

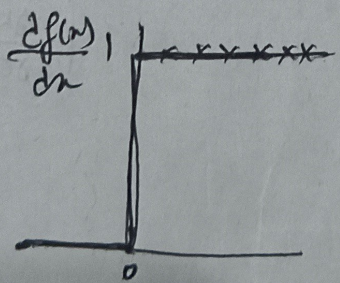
- derivative ReLU = 1; W will update
  = 0; Dead Neuron

Adv → Solving gradient vanishing
Postive max(0, x)

dis → Dead Neuron

⇔ Relu (Leaky) And Parametric Relu



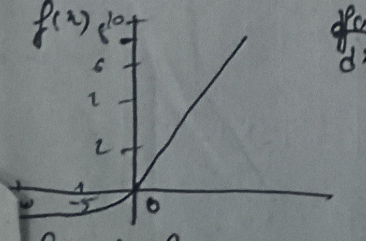


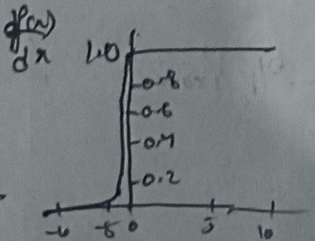
$$f(x) = \max(0.01x, x)$$

α = hyperparameter usually

Adv → no dead neuron
dis → not zero centric

⇔ Exponential Linear Units





$$f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha(e^{x}-1), & \text{else} \end{cases}$$

Adv → Zero Centered
no dead neuron

dis → Computation ++

Polynomial (non-linear data) | Linear (H/W) multi " (coordinate) fix-not-fix

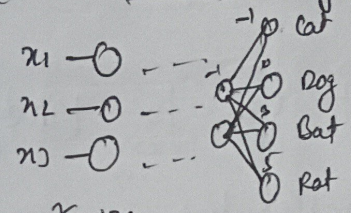
Frontend-fundamentals

Machine-Learning

⇐ Softmax function (Used in multiclass classification)

$$\text{Softmax} = \frac{e^{y_i}}{\sum_{k=0}^{n} e^{y_k}}$$

Ex

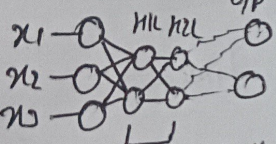


$\text{Cat} = \frac{e^{-1}}{e^{-1}+e^{0}+e^{3}+e^{5}} = 0.013$ $\text{Bat} = \frac{e^{3}}{e^{-1}+e^{0}+e^{3}+e^{5}} = 20.5$

$\text{Dog} = \frac{e^{0}}{e^{-1}+e^{0}+e^{3}+e^{5}} = 0.05$ $\text{Rat} = \frac{e^{5}}{e^{-1}+e^{0}+e^{3}+e^{5}} = 20.17$

$\text{Prob(Rat)} = \frac{0.5}{0.013+0.5+0.05+.17} \approx 17.1$

⇐ Which activation function to use when



2 classification → Sigmoid

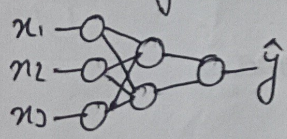
multi classification → Softmax

Relu & its variants

Tanh or sigmoid cause gradient vanishing

⇐ Loss & Cost function

Ex



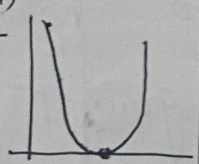
Loss fun()	MSE $(y-\hat{y})^2$
Cost fun()	$\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$

Calc ŷ in fp & points & Bp & points

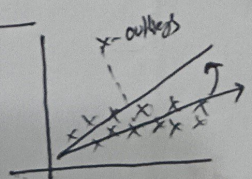
Calc Σŷ in fp & points & Bp update weights only once.

⇐ Types of Loss function (Regression)

1) MSE loss → $(y-\hat{y})^2$ Cost → $\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$

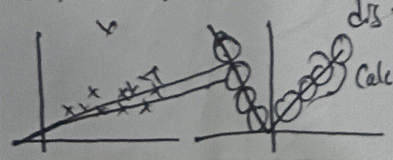


Adv	MSE is differentiable @ point includes 1 local or global Minima Converges faster
dis	Not robust to errors i.e. slope updates for outliers



2) MAE loss → $|y-\hat{y}|$ Cost → $\frac{1}{n}\sum_{i=1}^{n}|y_i-\hat{y}_i|$

Adv	Robust to outliers
Dis	Converges takes time



3) Huber loss = MSE + MAE

$$\text{Cost fn} = \begin{cases} \frac{1}{n}\sum_{i=1}^{n}(y-\hat{y})^2 & \text{if } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{else} \end{cases}$$

No outliers

outliers

Threshold

4) RMSE

$$\text{Cost fn} = \sqrt{\frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{N}}$$

Adv → Some units as o/p

dis → Sensitive to outliers

⟺ Loss or Cost function (classifications)

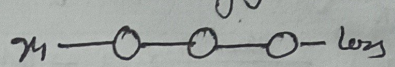
classification (Cross Entropy) ←
- ()₂ Cross Entropy | ()₂
- Categorical " " | multiclass
- Sparse " " | multiclass

1) Binary Cross Entropy

$Loss = -y * \log(\hat{y}) - (1-y) * \log(1-\hat{y})$

$Loss = \begin{cases} -\log(1-\hat{y}) & \text{if } y=0 \\ -\log(\hat{y}) & \text{if } y=1 \end{cases}$ Using Sigmoid

2) Categorical Cross Entropy



f1	f2	f3	o/p	j=1 Good	j=2 Bad	j=3 Neutral
—	—	4	good	1	0	0
—	—	7	bad	0	1	0
—	—	10	neutral	0	0	1

$Loss = L(x_i, y_i) = -\sum_{j=1}^{C} y_{ij} * \ln(\hat{y}_{ij})$

Using ONE, (C = Category)

$y_{ij} = \begin{cases} 1 & \text{if element } i \text{ in class } j \\ 0 & \text{else} \end{cases}$

$\hat{y_{ij}}$ = Using Softmax Act fun()

3) Sparse Categorical Cross Entropy

Similar to Categorical Cross Entropy except outputs max(prob)

ds → loosing info. (other prob lower)

⟺ Activation function & Combination with Loss functions

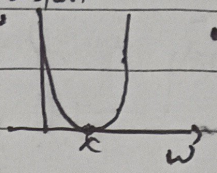
	Hidden Layers	Output layers	Problem Solved	Loss function used
1)	Relu or variants	Sigmoid	()₂ classificn.	()₂ CE
2)	Relu or variants	Softmax	multi "	Categorical or Sparse CE
3)	Relu or variants	Linear	Regression	MSE, MAE, Huber, RMSE

Topic..... Date.....

Optimizers

1) Gradient Descent

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$



- Epoch
- Iteration

Adv: Convergence True

dis: huge RAM req

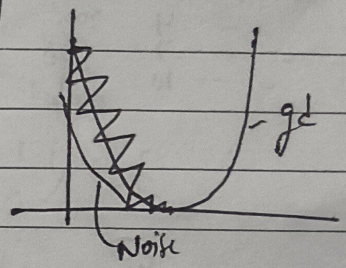
Ex: Using MSE

1 Epoch: 1000 → $\hat{y}$ → cost func
2 " : $\hat{y}$ → " "
n Epoch: $\hat{y}$ → " "

} low better gradient descent

let 100 Epoch done, $\frac{1000}{10}$ datapoints ⇒ 100 Epoch
↳ # iteration on each epoch
Total iteration = 100×1000

- Noise



2) Stochastic Gradient Descent

iteration = 1×1000
(one iteration = 1 Epoch)

Epoch = 1000
datapoints = 1000

Adv → Saving resource req

dis → Time ↑ Complexity & Noise +
Convergence time ++

3) Mini Batch SGD

- Batch datapoints = 100000

iteration×1 → 1000 Batch

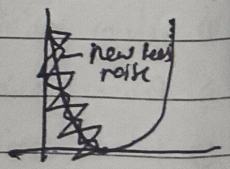
iteration×100 → 1000 Batch

Epoch=100, Iteration=100, datapoints=100000

Adv → Convergence time −, Noise −, resource req −−

dis → Noise E

Note → Smoothing noise



4) SGD with Momentum

- Noise − by smoothing noise

$$W_t = W_{t-1} - \eta \frac{\delta L}{\delta W_{t-1}}$$

let time $t_1\ t_2 \dots t_n$
val $a_1\ a_2 \dots a_n$

- β = 0 to 1

$V_{t_1} = a_1$, $V_{t_2} = \beta * V_{t_1} + (1-\beta) * a_2$
$= 0.95 * a_1 + (0.05) * a_2$

$V_{t_3} = \beta * V_{t_2} + (1-\beta) * a_3$
$= 0.95[0.95 * a_1 + (0.05) a_2] + (0.05) * a_3$

} exponential

Adv → Convergence time −−
noise −−

4) Adagrad optimizers (Adaptive gradient Descent)

- Learning rate updation (0.001 → dynamic)

$$W_t = W_{t-1} - \eta' \frac{\partial L}{\partial W_{t+1}} \qquad \eta' = \frac{\eta}{\sqrt{\alpha_t + \epsilon}} \qquad \eta' \propto \frac{1}{\alpha_t}$$

dis → possibility $\eta' \cong 0$ / $W_t \cong W_{t+1}$

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{\partial L}{\partial W_t}\right)^2$$

5) Adadelta & RMSPROP

$$\eta' = \frac{\eta}{\sqrt{Sdw + \epsilon}} \qquad Sdw_t = 0$$

$$Sdw_{t+1} = \beta * Sdw_t + (1-\beta)\left(\frac{\partial L}{\partial W_{t-1+1}}\right)^2$$

- Dynamic Learning Rate
- Smoothy EWA

6) Adam optimizers

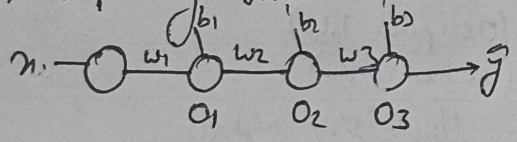
= SGD with momentum + RMSPROP

[Dynamic η + Smoothy] $\eta' = \frac{\eta}{\sqrt{Sdw_t + \epsilon}}$

$$W_t = W_{t-1} - \eta' Vdw \qquad Vdw_t = \beta * Vdw_{t-1} + (1-\beta)\frac{\partial L}{\partial W_t}$$

$$b_t = b_{t-1} - \eta' Vdb \qquad Vdb_t \text{ similarly}$$

⟺ Exploding Gradient Problem

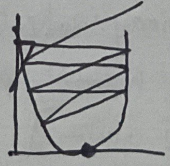


$$W_{inew} = W_{iold} - \eta \left[\frac{\partial L}{\partial W_{iold}}\right] \qquad \frac{\partial L}{\partial W_{iold}} = \frac{\partial L}{\partial O_3} \times \left[\frac{\partial O_3}{\partial O_2}\right] \times \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial W_{iold}}$$

$$\frac{\partial O_3}{\partial O_2} = \left[\frac{\partial \sigma(z)}{\partial z}\right] * \frac{\partial z}{\partial O_2}$$

$$= \left[\sigma(1-\sigma)\right] * \frac{\partial (O_2 * W_3 + b_3)}{\partial O_2} = (0-0.25) * W_3$$

let W_3 = very big ∴ will be very big, result never reaches gobal



⟺ Weight Initializing Technique

1) Uniform distribution

$W_{ij} \approx$ range $\left[\frac{a}{\sqrt{\text{input}}}, \frac{b}{\sqrt{I/P}}\right]$

2) Xavier/Glovot Initialization

a) Xavier Normal Initialization

$W_{ij} \approx$ Normal $(0, \sigma)$, $\sigma = \sqrt{\frac{2}{I/P + O/P}}$

b) Xavier Uniform Distribution

$W_{ij} \approx$ Uniform Distribution $\left[\frac{-\sqrt{6}}{\sqrt{I/P + O/P}}, \frac{\sqrt{6}}{\sqrt{I/P + O/P}}\right]$

3) Kaiming He Initialization

a) He Normal

$W_{ij} \approx (0, \sigma)$, $\sigma = \sqrt{\frac{2}{I/P}}$

b) He uniform d

$W_{ij} \approx$ Ud $\left[\frac{-\sqrt{6}}{\sqrt{I/P}}, \sqrt{\frac{6}{I/P}}\right]$

Key points

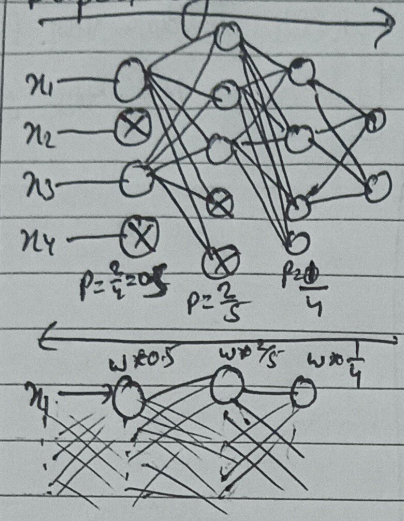
a) Weight should be small

b) Weight should not be same

c) Weight should have good variance

Topic..... Date.....

DropOut Layer



Overfitting occur similarly as ML.
Solution → [break chain using a val {p} ∈ (0 to 1)]
For forward propogation (Drop chain)
For Backward Propogation (multiply with w*p)

CNN ; Convolution Neural Network

- Human brain have 14 parts | 1 → visual area (Occipital lobe)
  12 → Rest part (Cerebral cortex)
- ANN ; Supervised ML [classification / regression] | I/P (Dataset) O/P (values, Prediction)
- CNN ; I/P (Images) O/P (Image classification, Obj detection Segmentation)
- Cerebral & Visual Cortex

  Visual Cortex (V1–V5) [Region (brain): I/P, integrates, process ; visual info (O/P of retina)]

  V1 → [Primary visual Cortex : (Orientation, edges & line)]

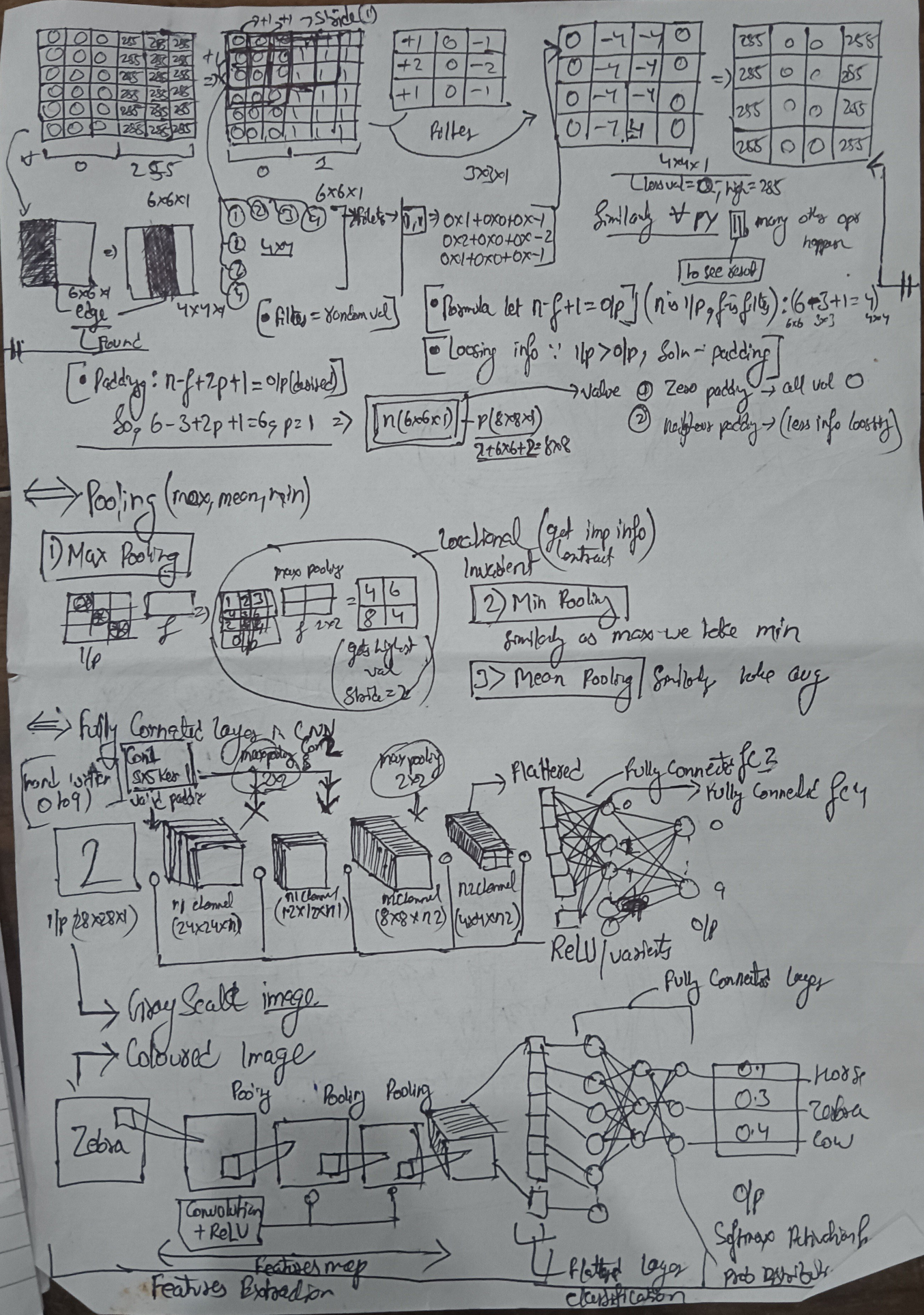
  V2 → [diff in colours, patterns, etc]

  V3 V4 V5 → [obj detection]

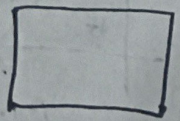
Convolution Operation In CNN

1) Normalization : divide 1px by 255
2) Filter applying
3) Relu is used : to obtain derivative 1px in back tracking
4) Pooling (max, mean, min)

[Single Convolution layer]

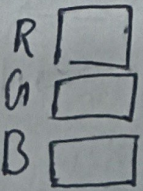

Filter
3x3x1
6x6x1
4x4x1
Similarly for RY
many other ops happen
to see result
Formula let n-f+1=o/p
Losing info :: i/p > o/p, Soln - padding
Padding : n-f+2p+1=o/p (desired)
So, 6-3+2p+1=6, p=1
Value ① Zero padding → all val 0
② neighbour padding → (less info loosty)
Pooling (max, mean, min)
1) Max Pooling
Locational Invariant (get imp info / extract)
2) Min Pooling
Similarly as max we take min
3) Mean Pooling Similarly take avg
Fully Connected Layer & CNN
Flattered
Fully Connected fc2
Fully Connected fc4
ReLU / variety
Gray Scale image
Coloured Image
Pooling
Zebra
Convolution + ReLU
Features map
Features Extraction
Flatten Layer
classification
Fully Connected Layer
Horse
Zebra
Cow
Softmax Activation
Prob Distribution

⇔ Images

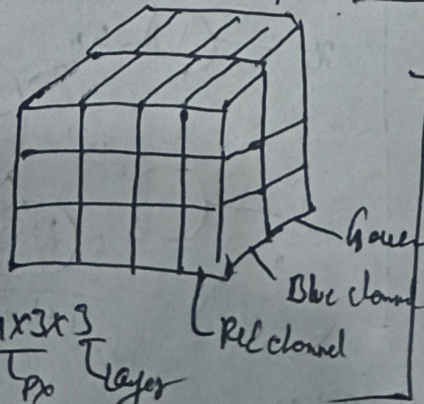


Colored Image

⇒



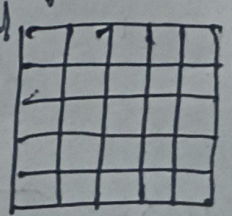
⇒



4x3x3
Px layer

[∀Px → 0 to 255]

RGB layered cross section of an image



5x5x1

[Px → 0 ~ 1]

→ Gray scale image

0 → Black

1 → White